Understand Async Job Systems

Async jobs simplify our application by letting us extract side effects out of the main path of our code. This can dramatically improve the performance of our application in a variety of situations. We don't have to worry about whether we should make the expensive HTTP call in our controller. We can simply fire a background job and let it process!

A good async job system will do much more than this. Complex tasks can be broken into subtasks and orchestrated to solve difficult problems with less code than if you built it yourself.

Let's start by looking at use cases for async jobs, followed by a litmus test for when to extract code into an async job. We'll finish this section by considering what makes a good job system.

Async Job Use Cases

Async jobs can be used for a variety of different tasks. On one hand, we have extremely simple tasks like sending emails. On the other, we can build entire data pipelines with fan-out fan-in style processing.

As you're reading these use cases, it's important to consider that async jobs execute as fast as they can, but they are not time-guaranteed. For example, we may expect that enqueuing an email will send it immediately, but the job system could be backed up and the email may take seconds, minutes, or even longer to process.

Let's break down some common use cases:

Send emails after a request (basic)

Emails are commonly executed asynchronously because they (almost always) require sending requests to external systems to deliver. Those systems could be slow or experience an outage. If we're implementing a "new user signup flow," then we may want to immediately create the user in our database, but we'll be okay if the welcome email is delivered behind the scenes.

Inform another system of an operation (basic)

An operation happens in our system, and we want to let another system know over an API. We could do that inside of the operation request itself, but we'd encounter the same potential problems as with sending an email. Making the request inside of an async job gives us a more stable and scalable solution.